

Predictive DCI Saturation Mapper

Startup & Pipeline Run Summary

Session date: 2026-09-23

This document summarizes the steps taken to restart, verify, and run the Predictive DCI Saturation Mapper pipeline end-to-end after a WSL restart. The project itself (source code, tests, docs) lives at `C:\Users\super\Projects\Predictive-DCI-Saturation-Mapper`; this write-up is exported separately as a record of the startup session.

1. Post-Restart Environment Check

After WSL was shut down and restarted, verified nothing was broken before proceeding:

- Confirmed the Python virtual environment (`.venv`, Python 3.14.4) was intact with all dependencies present.
- Ran the test suite: `pytest -q` → all 31 tests passed.
- Checked git status: working tree clean, still on master at the Stage 4 commit (d5e3070).
- Checked Docker: daemon was running, but no containers were up (expected, since a host restart stops all containers).

What this does: restarting WSL tears down every running process and container, but leaves the filesystem (code, virtual environment, git history) untouched. This step is a quick health check to confirm nothing on disk was corrupted and that the toolchain still works before doing any real work — it does not change anything.

2. Starting the Kafka Stack

Brought up the local Kafka broker (KRaft mode, no ZooKeeper) and Kafka UI defined in `docker-compose.yml`:

```
docker-compose up -d
```

Pulled fresh images (`apache/kafka:3.7.0` and `provectuslabs/kafka-ui:latest`) since the containers were newly created. Polled `docker inspect` until the `dci-kafka` health check reported healthy.

Service	Container	Port	Status
Kafka broker	dci-kafka	localhost:9092	healthy
Kafka UI	dci-kafka-ui	localhost:8080	running

What this does: the pipeline's stages talk to each other over Kafka, a message broker that decouples the telemetry producer (the generator) from the consumer that persists it. Kafka UI is just a web console for inspecting topics/messages; it's optional but useful for debugging. Nothing downstream can run until the broker reports healthy.

3. How Telemetry Works

There is no real network to poll, so `src/generator/circuit_profiles.py` defines four synthetic circuits, each a (site A, site B) pair with a capacity and a *trend archetype* — a formula for how its utilization moves over simulated time. The archetypes are deliberately chosen to stress the forecaster in different ways:

Archetype	Behavior	Why it's included
linear_ramp	Steady organic growth	The easy case — a straight-line fit should predict it well
step_change	Flat, then a sudden jump	Tests whether the model reacts fast after a regime change instead of averaging it away
seasonal_growth	Slow upward trend + daily/weekly cycles	Tests separating real trend from cyclical noise
healthy_plateau	Flat, just noise	A control circuit that should never get flagged

From utilization curve to wire format

At each tick, the generator reads a circuit's target utilization off its trend curve (plus random noise) and turns that percentage back into byte counters, the way a real router's sFlow/NetFlow/IPFIX export would report it: $\text{peak_bps} = \text{capacity_bps} \times \text{utilization\%}$ for whichever direction (in/out) is leading, and the other direction gets a randomized 55–90% of that value, alternating which side leads so the traffic isn't artificially symmetric. Both get converted into octet counts over the sample's time window and published as one JSON message — circuit id, site pair, link type, timestamp, interval, in/out octets, and capacity — to the Kafka topic `dci.telemetry.raw` (or printed to stdout with `--dry-run`, no broker needed).

From wire format back to a percentage

The consumer subscribes to that topic, and for each message: parses and validates it (`src/ingestion/parser.py`) — a malformed sample is rejected and logged rather than stored; computes utilization as the *peak* direction's bits-per-second divided by the circuit's capacity ($\text{max}(\text{in_bits}, \text{out_bits}) / \text{interval_s} / \text{capacity_bps}$); and writes the validated sample into SQLite via `TimeSeriesStore`. A sample at or above 85% prints with a `--WATCH` marker in the console as a live preview of what the alerting stage (Stage 4) does properly. This ingestion step doesn't judge whether a circuit *will* saturate — it just gets clean, ordered history into storage for the forecaster to work with.

What this does: together, the generator and consumer stand in for a real telemetry pipeline — router export → collector → time-series store. The one deliberate shortcut: simulated time runs much faster than wall-clock time (`--sim-hours-per-tick` vs `--tick-seconds`), so weeks of trend unfold in seconds. To keep that compression from corrupting the forecaster's math, the trend model's time axis is built from each sample's *declared* interval (`sample_interval_s`), not from diffing real arrival timestamps — so the fitted trend reflects what the telemetry claims about elapsed time, independent of how fast the demo actually ran.

4. Running the Pipeline End-to-End

With Kafka up, ran all four pipeline stages against a fresh `data/telemetry.sqlite`:

Stage 1 — Generator & Consumer

Started the SQLite-backed consumer in the background, then ran the synthetic telemetry generator against the local broker to stream simulated utilization data for 4 circuits (steady climb, seasonal, step-change, and

near-saturation profiles).

```
python -m src.ingestion.consumer --db-path data/telemetry.sqlite & python -m  
src.generator.telemetry_generator --bootstrap-server localhost:9092 \ --max-ticks 200  
--tick-seconds 0.2
```

Result: 824 telemetry samples ingested into SQLite (206 per circuit).

What this does: the generator stands in for a real sFlow/NetFlow collector — it fabricates plausible utilization readings for 4 circuits, each following a different traffic pattern, and publishes them to the Kafka topic. The consumer subscribes to that topic and writes every validated sample into SQLite, building the historical record everything else reads from. This is the only step where the environment is time-compressed (simulated weeks pass in seconds) so a full trend is visible without a real wait.

Stage 2 — Trend Forecasting

```
python -m src.modeling.forecaster --db-path data/telemetry.sqlite
```

Fit a recency-weighted trend per circuit and projected saturation dates. Output flagged three of four circuits as heading toward saturation, including one within hours.

What this does: reads each circuit's stored history and fits a recency-weighted linear trend (recent samples count more, so a step-change isn't diluted by older, flatter data). From that trend it extrapolates forward to estimate when utilization will cross 100% — the predicted saturation date — along with a confidence score based on how well the trend fits.

Stage 3 — Augment-Plan Cross-Check

```
python -m scripts.generate_demo_augment_plan --db-path data/telemetry.sqlite python -m  
src.planning.cross_check --db-path data/telemetry.sqlite
```

Generated a demo capacity-augment plan and compared each circuit's predicted saturation date against its planned upgrade date. All four possible outcomes were reproduced:

Circuit	Result	Detail
DCI-ASH-DAL-W1	AT RISK	Saturates 14.0 days before its planned upgrade
DCI-LON-FRA-W1	OK	Not on track to saturate
DCI-NYC-CHI-W1	OK	Planned upgrade lands 0.4 days ahead of saturation
DCI-SJC-PDX-W2	MISSING	Predicted to saturate, but no augment plan exists

What this does: this is the step that turns a forecast into a scheduling decision. The demo script fabricates a capacity-augment plan (upgrade dates) for some circuits, the way a real network planning tool would export one. The cross-check then compares each circuit's predicted saturation date from Stage 2 against its planned upgrade date and buckets it into one of four outcomes: AT RISK (saturates before the upgrade lands), OK (upgrade arrives in time, or no saturation is predicted), or MISSING (saturation predicted but no upgrade is even planned).

Stage 4 — Alerting & Dashboard

```
python -m src.alerting.pipeline --db-path data/telemetry.sqlite \ --augment-plan-path  
data/augment_pipeline.csv \ --dashboard-path data/dashboard.html
```

Re-ran the Stage 2/3 cross-check, wrote a static HTML dashboard to `data/dashboard.html`, and sent 2 alerts for the flagged circuits (DCI-ASH-DAL-W1 AT RISK, DCI-SJC-PDX-W2 MISSING PLAN). The dashboard was opened locally via the Windows default browser.

What this does: this is the delivery step — it re-runs the Stage 2/3 logic so the output is self-contained, renders a static HTML dashboard summarizing every circuit's status (so a human can check it without running any code), and pushes one alert per circuit that came back AT RISK or MISSING, so the people responsible for capacity planning are notified rather than having to go looking for the problem.

5. Outcome

- Environment verified healthy after the WSL restart (venv, tests, git, Docker).
- Kafka broker and Kafka UI brought up and confirmed healthy.
- Full 4-stage pipeline (generate → ingest → forecast → cross-check → alert/dashboard) executed successfully against fresh synthetic telemetry.
- Dashboard produced at `data/dashboard.html` and viewed in-browser.